Memory Allocation in Redux:

Redux stores all your app's global data in one big **JavaScript object. This** object lives in the browser's **memory heap**.

This object changes (updates) every time you dispatch (send) an action.

When the Redux Store is Created, memory is used for:

- **Initial State Tree** – This is the default data of your app, stored in nested objects.
- **Reducers** – Functions that know how to update the state.
- **Middleware Pipeline** – Functions that do extra work like logging or handling async logic.
- **Subscription Functions** – Code that runs whenever the state changes (like UI updates).

When You Dispatch an Action:

- Middleware and reducers run inside the call stack.
- A new version of the state is created and stored in memory

What happens to the old state?

- If some parts didn't change, they are reused by reference.
- If parts were updated, they are replaced (because Redux follows immutability).

Old or unused memory is cleaned up by JavaScript's Garbage Collector.

Memory problems can happen when:

- You store very large or deeply nested state data.
- Redux DevTools keeps too much history of actions and states.
- You forget to unsubscribe listeners or subscriptions.

Redux Memory Areas

Memory Area	What It Stores / When It's Used
Heap	• **State tree** (entire Redux store) • **Reducers** (functions stored in memory) • Middleware functions • **Subscriber/listener functions** from store.subscribe() • DevTools history snapshots
Call Stack	• Redux middleware runs • Reducers run • The new state is calculated

What Happens When Redux Store Is Created?

Creating a store (configureStore / createStore) allocates:

Heap Memory Allocations

Component	Memory Allocated For
Initial state	Entire initial state tree object
Reducer functions	References + closures
Middleware chain	Array of functions
Listeners	Subscriber array (initially empty)
DevTools enhancer	Internal tracking objects

Redux Toolkit uses:

- **Immer** (to produce immutable updates)
- **Proxy objects** (Drafts)

- **Patches** (for DevTools)

So memory is used for:

- Draft proxies
- Final nextState
- Patch logs

When You Dispatch an Action — What Happens in Memory?

1) dispatch() enters Call Stack

Temporary memory only.

2) Middleware executes

Each middleware gets:

- getState() reference
- dispatch reference

No new memory unless:

- Middleware logs data
- Middleware stores something internally
- DevTools stores a snapshot

3) Reducer Runs

Reducers are pure functions.

They receive:

(prevState, action)

But they **never mutate** prevState.

4) New State is Created (Immutability)

Redux always creates a *new object*, but NOT the whole tree.

This uses **structural sharing**:

- **Only updated branches get new memory.**
- **Unchanged branches reuse references.**

Example:

```
state = {
  user: { ... },
  posts: { ... },
  theme: { ... }
}
```

If only posts changes:

- posts → new object (new memory)
- user & theme → reused (no new memory)

What Happens to the Old State? (GC Internals)

When a new state is created:

- Old state becomes **unreferenced**
- JS Garbage Collector eventually removes it

BUT it only gets cleaned when:

- No part of the app holds references
  (example: DevTools / closures can keep old versions alive!)

Redux internally keeps these objects:

```
store = {
  dispatch: fn,
  getState: fn,
  subscribe: fn,
  replaceReducer: fn,
  [Symbol(observable)]: fn,

  __listeners: [fn, fn, ...],
  __nextListeners: [fn, fn, ...],

  __currentReducer: rootReducer,
  __currentState: stateTree,
}
```

Major memory contributors:

- __currentState (main heap object)
- __listeners (grows if not unsubscribed)
- DevTools connections